Instalación, configuración y despliegue

Este documento describe los requisitos, el procedimiento de puesta en marcha y las alternativas de publicación del prototipo. Está dirigido a cualquier persona técnica que necesite reproducir el entorno, incluido el equipo de evaluación.

---

1. Requisitos

Componente	Versión mínima	Observación
Python	3.10	Verificado en 3.11.15
Sistema	Windows, macOS o Linux	Sin dependencias del sistema
Espacio en disco	~400 MB	Principalmente el entorno de Python
Data Pack	7 archivos (D1–D7)	**No se redistribuye.** Lo entrega la organización
Clave de API	Opcional	Sin ella el sistema opera en modo respaldado

No se requiere base de datos, servidor de aplicación ni servicio externo.

2. Obtención del Data Pack

El Data Pack es provisto por la organización del desafío a través de la plataforma oficial y **no se incluye en este repositorio**. Una vez descargado, los siete archivos deben ubicarse en el directorio `data/`:

```
data/
├── D1_wellbeing_survey.csv
├── D2_support_services.csv
├── D3_academic_trajectory.csv
├── D4_testimonials.json
├── D5_help_line_conversations.json
├── D6_services_map.geojson
└── D7_calendar.csv
```

Si falta cualquiera de los archivos obligatorios, la aplicación se detiene en el arranque con un mensaje que identifica exactamente cuál falta, en lugar de fallar más adelante con un error no descriptivo.

3. Instalación

3.1 Procedimiento automático

El repositorio incluye un instalador que comprueba el entorno, crea el entorno virtual, instala las dependencias y verifica la configuración. **Es el procedimiento recomendado para el equipo.**

Sistema	Archivo	Ejecución
Windows	`instalar.bat`	Doble clic sobre el archivo
macOS / Linux	`instalar.sh`	`chmod +x instalar.sh && ./instalar.sh`

Ambos instaladores verifican la presencia de los siete archivos del Data Pack y detienen la ejecución indicando cuál falta. También.info si la clave de API fue detectada.

3.2 Procedimiento manual

Se recomienda un entorno virtual aislado.

```
git clone https://github.com/sergio-beltran-abalos/aethera-m5.git
cd aethera-m5

python -m venv .venv

# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
```

4. Configuración de la clave de API

La clave **es opcional**. El sistema detecta su ausencia y opera con los respaldos descritos en la sección 7.

4.1 Proveedor predeterminado: Google Gemini

```
# Windows (CMD)
set GEMINI_API_KEY=su_clave

# Windows (PowerShell)
$env:GEMINI_API_KEY="su_clave"

# macOS / Linux
export GEMINI_API_KEY=su_clave
```

Para que persista entre sesiones en Windows:

```
setx GEMINI_API_KEY "su_clave"
```

La clave se obtiene en <https://aistudio.google.com/apikey>. El nivel gratuito del proveedor resulta suficiente para el volumen del piloto (véase `COSTOS.md`).

4.2 Proveedores alternativos

La capa de modelo es intercambiable. La variable `AETHERA_PROVEEDOR` determina el transporte y la clave asociada:

AETHERA_PROVEEDOR	Variable de entorno	Observación
`gemini` *(predeterminado)*	`GEMINI_API_KEY`	Nivel gratuito
`openai`	`OPENAI_API_KEY`	Requiere cuenta con facturación
`huggingface`	`HF_TOKEN`	Modelos abiertos mediante Inference API
`compatible`	`COMPATIBLE_API_KEY`	Endpoint compatible con la API de OpenAI; configurar `AETHERA_BASE_URL`
`offline`	*(ninguna)*	Desactiva el uso de modelos externos

Si el identificador del modelo predeterminado no se encuentra disponible para una cuenta determinada, puede sobrescribirse sin modificar el código:

```
set AETHERA_MODELO=otro_identificador_de_modelo
```

4.3 Verificación de la configuración

```
python app.py --diagnostico
```

La salida confirma el número de registros cargados de cada dataset, el proveedor y modelo activos, y si la clave fue localizada. **El valor de la clave nunca se imprime.**

5. Ejecución

```
python app.py
```

La interfaz queda disponible en <http://127.0.0.1:7860>.

Variables de entorno

Variable	Predeterminado	Función
`AETHERA_PROVEEDOR`	`gemini`	Transporte del modelo de lenguaje
`AETHERA_MODELO`	según proveedor	Identificador del modelo
`AETHERA_DATA`	`./data`	Ubicación alternativa del Data Pack
`AETHERA_INSTITUCION`	`UNI_NOVA_AETHER`	Institución a atender
`AETHERA_LANZAR_UI`	`1`	`0` desactiva el arranque de la interfaz
`AETHERA_SHARE`	`0`	`1` genera una dirección pública temporal

6. Estructura del repositorio

```
instalar.bat                Instalador automático para Windows
instalar.sh                 Instalador automático para macOS y Linux
app.py                      Lanzador; exigido en la raíz por Hugging Face Spaces
src/aethera/                Núcleo de la aplicación (16 módulos)
├── config.py               Constantes, supuestos y taxonomías
├── datos.py                Carga y preparación del Data Pack
├── segmentos.py            Reconstrucción de M5 y perfiles
├── horarios.py             Solver de compatibilidad horaria
├── servicios.py            Agenda, ranking y propuesta de citas
├── ml.py                   Modelos de asistencia y de necesidad
├── calendario.py           Eventos del calendario académico
├── guardrails.py           Guardrails de entrada y salida
├── ia.py                   Capa de modelo de lenguaje con respaldos
├── agente.py               Orquestación del turno
├── pulso.py                Registro de ánimo y consentimiento
├── bienestar.py            Agregados e indicadores
├── evaluacion.py           Experimentos reproducibles
├── vistas.py               Componentes HTML
└── ui.py                   Interfaz Gradio
docs/                       Documentación técnica y entregables
notebook/                   Cuaderno de análisis con salida ejecutada
tests/                      Verificaciones automatizadas
tools/                      Utilidades de conversión
```

7. Operación sin clave de API

El sistema está diseñado para no depender de un servicio externo. En ausencia de clave:

- La clasificación de necesidad la resuelve un clasificador **entrenado sobre D5**

(2 500 conversaciones etiquetadas), con una exactitud de validación cruzada de **80,8 %**,

frente a una línea base del 20,5 %.

- El planificador de turno aplica reglas deterministas que cubren las siete acciones.
- El redactor emplea plantillas construidas exclusivamente con datos verificados del Data Pack.

Las tres capas de guardrail, la agenda, la propuesta de citas, la vista de Bienestar y los

indicadores funcionan íntegramente. Esta capacidad se verifica de forma automatizada:

```
python tests/escenario_tercero.py
```

La comprobación confirma que el sistema ejecuta un turno completo —incluidos los casos de

riesgo y de privacidad— sin realizar una sola llamada a un modelo externo.

Limitación. El modo sin clave produce respuestas correctas pero de menor calidad

lingüística. Resulta adecuado para reproducir la funcionalidad y para la evaluación técnica;

no sustituye la experiencia con el modelo generativo en la demostración.

8. Reproducción de resultados

```
python -m aethera.segmentos      # reconstrucción del embudo M5
python -m aethera.ml             # modelos, métricas y límites
python -m aethera.evaluacion     # los tres experimentos y el análisis de sensibilidad
```

La agenda de turnos se genera con una **semilla fija**, de modo que la demostración produce números idénticos en cada ejecución. Esta propiedad es deliberada: garantiza que el equipo de evaluación observe exactamente lo que observó el equipo autor.

Verificaciones automatizadas

Ocho suites, todas ejecutables sin clave de API y sin conexión:

```
python tests/verificar_e2.py        # 78 comprobaciones de extremo a extremo
python tests/verificar_handlers.py  # cada handler de la interfaz, con entradas reales
python tests/verificar_ui.py        # la aplicación Gradio construye sin errores
python tests/revisar_html.py        # todas las vistas generan HTML bien formado
python tests/revisar_docs.py        # integridad textual de docs y código
python tests/revisar_enlaces.py     # enlaces, estructura exigida y formato de los PDF
python tests/escenario_tercero.py   # un tercero sin clave puede correr el repositorio
python tests/auditar_publicacion.py # nada publicable filtra información sensible
```

Para ejecutarlas todas de una vez:

```
python tests/verificar_e2.py && python tests/verificar_handlers.py && \
python tests/verificar_ui.py && python tests/revisar_html.py && \
python tests/revisar_docs.py && python tests/revisar_enlaces.py && \
python tests/escenario_tercero.py && python tests/auditar_publicacion.py
```

9. Opciones de publicación

El prototipo se publica sin tarjeta de crédito y sin costo. Las dos vías verificadas durante el desarrollo se describen en detalle en `PUBLICACION.md`.

Opción	Coste	Pide tarjeta	Instalación extra	Verificada
`compartir.bat` / `.sh`	0 USD	No	Cloudflare Tunnel	**Sí**
`AETHERA_SHARE=1`	0 USD	No	Ninguna	**Sí**

Hugging Face Spaces fue evaluada y descartada: aunque su nivel gratuito no genera cargo, la creación del Space solicita una tarjeta de crédito como verificación de identidad.

Vía rápida: enlace de Gradio

```
set AETHERA_SHARE=1        # Windows (CMD)
export AETHERA_SHARE=1     # macOS / Linux
python app.py
```

La salida muestra la dirección `https://<codigo>.gradio.live`. No requiere instalar nada.

Vía recomendada: túnel de Cloudflare

El script `compartir.bat` (Windows) o `compartir.sh` (macOS/Linux) automatiza el proceso completo: verifica el entorno, comprueba el Data Pack, instala `cloudflared` si no está presente, inicia la aplicación y muestra la dirección `https://<aleatorio>.trycloudflare.com`.

Procedimiento manual, si se prefiere ejecutarlo por separado:

```
winget install --id Cloudflare.cloudflared     # Windows
brew install cloudflared                       # macOS

python app.py                                    # en una terminal
cloudflared tunnel --url http://127.0.0.1:7860 # en otra
```